Supplementary Material: A Differentiable Atari VCS: A Complex, Fully Known Ground Truth for Explainable AI

Anonymous submission

This supplement collects the analyses behind the main paper. We give the full proofs of the two theorems; a numerical study of how the relaxation parameters α and T shape the gradient; an empirical and analytic account of when the *fully relaxed* forward stays bit-exact (a cast-margin model, its (α, T) likelihood map, and boundary measurements on the full simulator), accompanied by the beam-timed comparison videos; and the implementation-effort timeline referenced in the Results. It is self-contained: we first restate the soft/hard formulation, prove the forward-equivalence and temperature-limit results, then study the relaxation gradient and forward bit-exactness, and finally show the effort timeline.

1 Setup and Notation

The differentiable emulator runs one of two transition maps over the full VCS state $x = (\text{registers}, \text{RAM}, \text{RIOT}, \text{TIA}, \text{frame buffer})$: the bit-exact *hard* map Φ_{H} and the differentiable *soft* map Φ_{S}. In both, the handler that runs is the one for the opcode byte at the program counter pc. The handlers are assembled from five primitives, which we restate here so that the proofs are self-contained.

The first primitive is the memory read. A read of a memory tensor $r \in \mathbb{R}^M$ of size M (the ROM or the RAM) at an integer address $a \in \{0, \dots, M-1\}$ is written as a dot product against the one-hot address indicator $\mathbf{1}_a$,

$$\mathrm{peek}(r, a) = \mathbf{1}_a^\top r = \textstyle\sum_i [\![i = a]\!]\, r_i = r_a, \qquad (1)$$

so the forward value is the array element r_a, while the gradient with respect to r is the one-hot vector $\mathbf{1}_a$.

The second primitive is opcode dispatch. With a score vector (logits) $\ell \in \mathbb{R}^K$ over the $K = 256$ opcodes, a temperature $T > 0$, and the softmax weights $w = \mathrm{softmax}(\ell/T)$, the relaxed dispatch over the candidate handler-output rows V is the convex combination

$$\mathrm{select}(\ell, V; T) = w^\top V, \qquad w_k = \frac{e^{\ell_k/T}}{\sum_j e^{\ell_j/T}}. \qquad (2)$$

The third primitive is the conditional branch. For a relaxed status flag $f \in [0, 1]$ written as a logit $z = s(2f - 1)$, where the sign s selects branch-when-set or branch-when-clear, and a sharpness α, the gate and the relaxed program counter are

$$g = \sigma(\alpha z), \qquad \mathrm{pc}_{\mathrm{soft}} = (1 - g)\,\mathrm{pc}_{\bar{b}} + g\,\mathrm{pc}_b = \mathrm{pc}_{\bar{b}} + g\,\delta, \qquad (3)$$

where $\mathrm{pc}_{\bar{b}}$ is the fall-through (not-taken) counter and $\mathrm{pc}_b = \mathrm{pc}_{\bar{b}} + \delta$ is the taken counter for the signed branch offset δ.

The fourth primitive is the straight-through estimator. For any soft value that has a matching hard value it returns

$$\mathrm{STE}(\text{soft}, \text{hard}) = \text{soft} + \mathrm{sg}(\text{hard} - \text{soft}), \qquad (4)$$

where $\mathrm{sg}(\cdot)$ is the stop-gradient operator, so the forward value equals *hard* while the backward pass uses the gradient of *soft*.

The fifth primitive is the *relaxed read*, the differentiable counterpart of the exact read (1). It blurs the integer address with a distance-softmax of width T, returning the temperature-weighted average of the neighbouring cells,

$$\mathrm{peek}_{\mathrm{R}}(r, a; T) = \textstyle\sum_k w_k\, r_{a+k}, \qquad w_k = \frac{e^{-|k|/T}}{\sum_j e^{-|j|/T}}, \qquad (5)$$

so that $\mathrm{peek}_{\mathrm{R}} \to r_a$ as $T \to 0$ while the address carries a nonzero gradient for $T > 0$. The executed (SOFT-STE) path uses the exact one-hot read (1); only the fully relaxed variant of Theorem 2 uses (5).

In the executed soft step, dispatch uses the saturated (hard) form of (2) and the branch returns $\mathrm{STE}(\mathrm{pc}_{\mathrm{soft}}, \mathrm{pc}_{\mathrm{hard}})$. The relaxed forms (2) and (3) *without* the straight-through correction define the "fully relaxed" variant analysed in Theorem 2. The temperature-limit result needs one non-degeneracy assumption, identical to the one in the main paper.

Assumption 1 (Decision margins)**.** *Along the executed trajectory of length N, every discrete decision has a strictly positive margin. For opcode dispatch at step t with logits $\ell^{(t)}$ and winner $k_t^\star$, the logit gap $\Delta_t = \ell^{(t)}_{k_t^\star} - \max_{k \neq k_t^\star} \ell^{(t)}_k$ is positive, and for each conditional branch with flag logit z_t, the margin $m_t = |z_t|$ is positive. Let $\Delta_{\min} = \min_t \Delta_t > 0$ and $m_{\min} = \min_t m_t > 0$.*

These positive margins are a property of the *executed* length-N trajectory; geometrically they define a tube around it inside which the relaxed maps agree with the hard one. The relaxed read (5) needs no margin: its off-target weights decay unconditionally with T (proved below), so only dispatch and branches require Assumption 1.

The three execution modes referenced throughout are summarised in Table 1.

Table 1: The three execution modes. SOFT-STE is the mode used for every gradient in this paper: its forward pass is bit-identical to HARD (Theorem 1) and its backward pass is a surrogate. The fully relaxed mode (FULL) is used only for the temperature-limit analysis (Theorem 2); its forward pass is bit-exact only inside the corner of small T and large α.

Mode	Forward	Gradient	Used for
HARD	bit-exact	none	conformance
SOFT-STE	= HARD	surrogate	attribution
FULL	exact in corner	relaxed	$T\to 0$ study

Numerical scope. Soft mode keeps every quantity in float32 but only ever at integer values in $[-2^{24}, 2^{24}]$ (Theorem 1), so no representation error accrues and the equalities below are exact, not approximate. Both ports apply the *same* round-to-integer after each soft primitive, so JUTARI (Zygote) and JAXTARI (JAX/XLA) produce identical integer trajectories. The float path carries gradients only; a quantity outside the 24-bit range—which the VCS never produces—would forfeit the exactness guarantee.

2 Proofs

Theorem 1 (Exact forward equivalence)**.** *For every reachable state x and every finite sharpness α and temperature T, the soft and hard one-step maps are equal by design, $\Phi_{\mathrm{S}}(x) = \Phi_{\mathrm{H}}(x)$, with identical float32 representations. Consequently, over a trajectory of N steps, $x_t^{\mathrm{S}} = x_t^{\mathrm{H}}$ for all $t \le N$. The modes differ only in the Jacobian $\partial\Phi_{\mathrm{S}}/\partial x$, which is defined and generically nonzero where $\partial\Phi_{\mathrm{H}}/\partial x$ is zero or undefined.*

Proof. We first prove the one-step equality $\Phi_{\mathrm{S}}(x) = \Phi_{\mathrm{H}}(x)$ for an arbitrary reachable x, and then extend it to a trajectory of length N by induction on the step index.

One-step equality. Fix a reachable x. In both modes the handler that executes is selected by the decoded opcode byte through the same hard switch (the softmax form (2) is not evaluated on the forward path), so both modes run the same handler on the same input. It therefore suffices to show that each primitive the handler uses returns the same forward value in both modes. A memory read at the integer address a uses an exact one-hot indicator, so

$$\mathrm{peek}_{\mathrm{S}}(r, a) = \mathbf{1}_a^{\top} r = \textstyle\sum_i [\![i = a]\!]\, r_i = r_a = \mathrm{peek}_{\mathrm{H}}(r, a), \tag{6}$$

the value a hard array read returns. Register, status-flag, binary-coded-decimal, and timer updates use the same integer and round arithmetic in both modes (soft mode merely keeps the results in float32). A conditional branch returns the straight-through value (4). Since sg is the identity in the forward direction,

$$\begin{aligned}\mathrm{STE}(\mathrm{pc}_{\mathrm{soft}}, \mathrm{pc}_{\mathrm{hard}}) &= \mathrm{pc}_{\mathrm{soft}} + \mathrm{sg}(\mathrm{pc}_{\mathrm{hard}} - \mathrm{pc}_{\mathrm{soft}}) \\ &= \mathrm{pc}_{\mathrm{hard}}\end{aligned} \tag{7}$$

for every α and T. Every component of the handler output thus equals its hard counterpart, and as the two handlers act on the same input,

$$\Phi_{\mathrm{S}}(x) = \Phi_{\mathrm{H}}(x). \tag{8}$$

These equalities hold in float32, not merely over the reals: IEEE-754 single precision has a 24-bit significand and so represents every integer in $[-2^{24}, 2^{24}]$ exactly (IEEE 2019; Goldberg 1991), and every VCS quantity lies in this range (data bytes ≤ 255, 13-bit addresses, per-frame cycle counts of a few tens of thousands), so (6)–(8) are exact.

Induction on the trajectory. Let the two runs share the initial state and evolve by $x_{t+1}^{\bullet} = \Phi_{\bullet}(x_t^{\bullet})$. We show $x_t^{\mathrm{S}} = x_t^{\mathrm{H}}$ for all $0 \le t \le N$.

Base case ($t = 0$): $x_0^{\mathrm{S}} = x_0^{\mathrm{H}}$ by the shared initial state.

Inductive step: assume $x_t^{\mathrm{S}} = x_t^{\mathrm{H}} =: x$ for some $t < N$. Applying the one-step equality (8) to x,

$$x_{t+1}^{\mathrm{S}} = \Phi_{\mathrm{S}}(x_t^{\mathrm{S}}) = \Phi_{\mathrm{S}}(x) = \Phi_{\mathrm{H}}(x) = \Phi_{\mathrm{H}}(x_t^{\mathrm{H}}) = x_{t+1}^{\mathrm{H}}, \tag{9}$$

where the outer equalities use the inductive hypothesis and the middle one is (8). By induction $x_t^{\mathrm{S}} = x_t^{\mathrm{H}}$ for all $t \le N$.

Gradients. The stop-gradient in (4) alters only the reverse-mode rule: the returned counter carries the derivative of $\mathrm{pc}_{\mathrm{soft}} = \mathrm{pc}_{\bar{b}} + g\,\delta$, namely $\alpha\, g(1-g)\,\delta$, which is generically nonzero where the hard branch—a step function of the flag—has zero or undefined derivative. Likewise the read (1) has Jacobian $\mathbf{1}_a$ with respect to r. The forward pass is unchanged while gradients become available. □

Theorem 2 (Temperature-limit bound)**.** *Consider the fully relaxed emulator that uses the softmax dispatch* (2)*, sigmoid branch* (3)*, and distance-softmax reads* (5) without *the straight-through correction. Under Assumption 1, its one-step map converges to the hard map as $T \to 0$ and $\alpha \to \infty$, with per-step deviation*

$$\begin{aligned}\big\|\Phi_{\mathrm{S}}^{T}(x) - \Phi_{\mathrm{H}}(x)\big\|_{\infty} \;\le\; C\big[&(K-1)\, e^{-\Delta_{\min}/T} \\ &+ e^{-\alpha m_{\min}} + \rho\, e^{-1/T}\big],\end{aligned} \tag{10}$$

where C bounds the value range (bytes ≤ 255, branch offsets ≤ 256) and ρ the number of reads in the step. Over N steps, with Lipschitz constant $L \ge 1$ for error propagation, the trajectory deviation is at most $\frac{L^N - 1}{L - 1}$ times the right-hand side of (10)*, which is $\mathcal{O}(e^{-c/T})$ with $c = \min(\Delta_{\min}, 1)$.*

Proof. We bound the deviation of the relaxed one-step map from the hard one and then propagate it along the trajectory. Throughout, C bounds the range of any state value (bytes ≤ 255, branch offsets ≤ 256).

Dispatch term. At a step with logits ℓ and winner $k^{\star}$ the gap is $\Delta \ge \Delta_{\min} > 0$ by Assumption 1, so for any loser $k \ne k^{\star}$,

$$w_k = \frac{e^{\ell_k/T}}{\sum_j e^{\ell_j/T}} \le \frac{e^{\ell_k/T}}{e^{\ell_{k^\star}/T}} = e^{(\ell_k - \ell_{k^\star})/T} \le e^{-\Delta_{\min}/T}, \tag{11}$$

so the non-winning mass is $1 - w_{k^\star} = \sum_{k \ne k^\star} w_k \le (K-1)e^{-\Delta_{\min}/T}$. As the dispatch output is the convex com-

bination $\sum_k w_k V_k$,

$$\begin{aligned}\Big\| \textstyle\sum_k w_k V_k - V_{k^\star} \Big\|_\infty &\le (1 - w_{k^\star}) \max_k \|V_k - V_{k^\star}\|_\infty \\ &\le (K-1)\, e^{-\Delta_{\min}/T}\, C.\end{aligned} \tag{12}$$

Branch term. For a margin $m = |z| \ge m_{\min}$ the gate error is

$$\big|\sigma(\alpha z) - [\![z > 0]\!]\big| = \sigma(-\alpha m) \le e^{-\alpha m_{\min}}, \tag{13}$$

so the blended counter $\mathrm{pc}_{\bar{b}} + g\,\delta$ differs from the hard target $\mathrm{pc}_{\bar{b}} + [\![z > 0]\!]\,\delta$ by

$$\big|(g - [\![z > 0]\!])\,\delta\big| \le |\delta|\, e^{-\alpha m_{\min}} \le C\, e^{-\alpha m_{\min}}. \tag{14}$$

Read term. A distance-softmax read (5) deviates from the hard value r_a by the neighbour pull. Using $|r_{a+k} - r_a| \le 2C$ and $1 - w_0 = \big(\sum_{k\neq 0} e^{-|k|/T}\big)/\big(\sum_j e^{-|j|/T}\big) \le 2\sum_{k\ge 1} e^{-k/T} = 2e^{-1/T}/(1 - e^{-1/T})$,

$$\Big|\textstyle\sum_{k\neq 0} w_k (r_{a+k} - r_a)\Big| \le (1 - w_0)\, 2C \le C\, e^{-1/T} \tag{15}$$

for T below a fixed T_0, absorbing the geometric factor and the byte range into C. Unlike dispatch and branch this requires no margin: the integer address spacing is ≥ 1, so the off-target weights decay as $e^{-1/T}$ unconditionally. The ρ reads of the step contribute at most $\rho\, C\, e^{-1/T}$.

Per-step bound. Adding (12), (14), and (15) bounds the one-step deviation: for every reachable x,

$$\begin{aligned}\delta_{\text{step}} &:= \big\|\Phi_{\mathrm{S}}^{T}(x) - \Phi_{\mathrm{H}}(x)\big\|_\infty \\ &\le C\big[(K-1)e^{-\Delta_{\min}/T} + e^{-\alpha m_{\min}} + \rho\, e^{-1/T}\big],\end{aligned} \tag{16}$$

which is (10).

Propagation. Let $L \ge 1$ be a Lipschitz constant of Φ_{H} in $\|\cdot\|_\infty$ and $e_t = \|x_t^{T,\mathrm{S}} - x_t^{\mathrm{H}}\|_\infty$ the trajectory error after t steps, with $e_0 = 0$. Each step adds at most δ_{step} to the hard map applied to the accumulated error,

$$e_{t+1} \le L\, e_t + \delta_{\text{step}}, \tag{17}$$

and unrolling from $e_0 = 0$ gives

$$e_N \le \delta_{\text{step}} \sum_{i=0}^{N-1} L^i = \delta_{\text{step}}\, \frac{L^N - 1}{L - 1}. \tag{18}$$

As $T \to 0$ and $\alpha \to \infty$, $\delta_{\text{step}} = \mathcal{O}(e^{-c/T}) \to 0$ with $c = \min(\Delta_{\min}, 1)$, so the relaxed trajectory converges to the hard one, $\Phi_{\mathrm{S}}^{T} \to \Phi_{\mathrm{H}}$. □

3 Effect of the Relaxation Parameters on the Gradient

Theorems 1 and 2 say the hard limit ($\alpha \to \infty$ for the sigmoid branch, $T \to 0$ for the softmax select) is exact in value but degenerate in gradient. We illustrate both knobs on JUTARI with a single target sprite (an invader) whose horizontal placement is produced by the soft primitives. For the sigmoid branch we plot the screen-space directional-derivative saliency $\partial(\text{screen})/\partial(\text{control})$ as the sharpness α is swept (Figure 1). The main paper additionally plots the gate and its gradient directly, and shows the softmax select's temperature effect in pixel space (a sampled sprite-occupancy heatmap). In the gradient maps the colour encodes the *sign* of the derivative—red where a pixel brightens as the control increases (the sprite arriving) and blue where it darkens (the sprite leaving)—and its intensity the magnitude, on a black background. The colour bar is in absolute units and the per-panel number is the peak relative to the row maximum.

For the branch the position is $\mathrm{pc} = (1 - g)\,\mathrm{pc}_L + g\,\mathrm{pc}_R$ with $g = \sigma(\alpha z)$, so its derivative with respect to the control z is $\alpha\,\sigma(\alpha z)\,(1 - \sigma(\alpha z))\,(\mathrm{pc}_R - \mathrm{pc}_L)$: a dipole, blue on the placement being vacated and red on the one being entered. The magnitude $\alpha\,\sigma(\alpha z)(1 - \sigma(\alpha z))$ is *non-monotonic* in α at the fixed operating point z=0.25. A small α gives a shallow gate and hence a small slope, and a large α saturates the gate ($\sigma(\alpha z) \to 1$, so $\sigma(1 - \sigma) \to 0$). The maximum is the intermediate α for which αz falls in the sigmoid's steep band, near α=6 here—$\alpha = 2, 6, 20$ give peak magnitudes $0.53, 1.00, 0.15$, and at α=20 the gate has essentially switched and the gradient has nearly vanished (the hard-branch limit). The main paper plots this gate and its gradient directly.

In every panel the forward render is the exact hard one (Theorem 1), and only the gradient changes. It is most informative at intermediate relaxation and decays to zero as the relaxation is sharpened toward the hard limit (Theorem 2). This is a qualitative study of how the parameter *values* act on the gradient, run on JUTARI, and the values need not be tuned for the bit-exact forward pass.

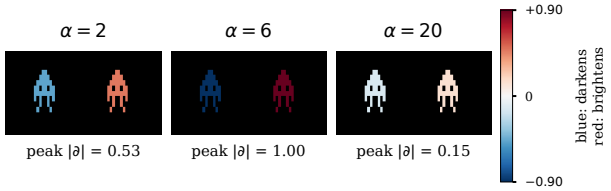

Figure 1: Soft branch: the screen-space gradient $\partial(\text{screen})/\partial z$ as the sharpness α is swept, on JUTARI. The target sprite's placement is a sigmoid-gated blend of a left and a right position, so the gradient is a dipole—blue where a pixel darkens (the placement being vacated) and red where it brightens (the one being entered). The colour bar is in absolute units (here ± 0.90) and the per-panel number is the peak relative to the maximum. The dipole is strongest at an intermediate α (here α=6) and nearly vanishes by α=20 as the gate saturates—the hard-branch limit (cf. Theorems 1 and 2).

4 Forward Bit-Exactness Under Full Relaxation

The gradient study above keeps the straight-through estimator, so the forward pass is bit-exact at any α and T (Theorem 1). To probe the temperature-limit bound (Theorem 2) empirically we instead *drop* the straight-through

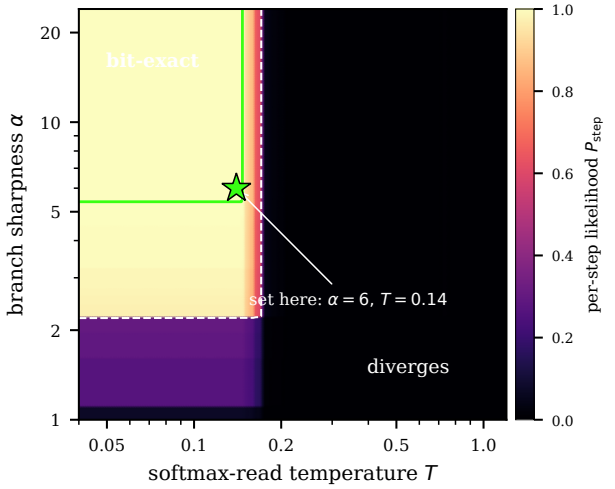

Figure 2: Overview of the per-step bit-exactness likelihood $P_{\text{step}} = p_{\text{read}}(T)^{\rho}\, p_{\text{branch}}(\alpha)^{f_b}$ of the fully relaxed forward across the (α, T) plane (Space Invaders). A bit-exact corner—bright, with the green contour marking P_{step}=1—sits at large branch sharpness α and small read temperature T, bounded by a gradual branch boundary (in α) and a sharp temperature boundary (in T); the white dashed line is P_{step}=0.5. Because P_{step} factorises, the plane is the outer combination of the two 1D profiles, and the two boundary sweeps of Table 2 are slices along the green contour's two edges. The star marks the recommended operating point (α=6, T=0.14)—the corner where they cross.

correction—the fully relaxed forward, where α and T act on the executed values themselves—and run it on the full JUTARI simulator executing the real Space Invaders ROM. Every instruction casts the sigmoid-blended program counter and the temperature-blended operand reads back to integers, so the trajectory stays bit-exact only while every such cast rounds to the value the executed (straight-through) path would produce: a large α keeps the rounded soft program counter on the hard branch target, and a small T keeps the distance-softmax read on the addressed byte.

Because each cast is a rounding, the likelihood of staying bit-exact follows from the cast margins. A crisp-flag branch with signed offset δ keeps its rounded soft program counter on the hard target exactly when $|\delta| < \tau_b(\alpha)$, $\tau_b(\alpha) = \frac{1}{2}(1 + e^{\alpha})$, so the per-branch exactness $p_{\text{branch}}(\alpha)$ is the fraction of executed branch offsets below τ_b. A temperature-T read at address a returns $\text{round}\big(\sum_k w_k \,\text{mem}[a{+}k]\big)$ with $w_k \propto e^{-|k|/T}$, and is exact when the neighbour pull $\big|\sum_{k\neq 0} w_k(\text{mem}[a{+}k] - \text{mem}[a])\big| < \frac{1}{2}$, giving a per-read exactness $p_{\text{read}}(T)$. Treating the ρ reads and the occasional branch of an instruction as independent casts, the per-step bit-exactness likelihood is $P_{\text{step}}(\alpha, T) = p_{\text{read}}(T)^{\rho}\, p_{\text{branch}}(\alpha)^{f_b}$, with $\rho = 2.16$ reads per instruction and branch fraction $f_b = 0.374$ measured on the executed Space Invaders trace, and the run is bit-exact only while every cast rounds correctly. P_{step} is a *heuristic* likelihood—it treats the casts as independent Bernoulli rounds, which they are not—so it predicts *where* exactness degrades rather than giving a probabilistic guarantee; the guarantee is the deterministic worst-case cast condition, with P_{step}=1 exactly when every cast margin holds.

Table 2: Forward bit-exactness of the *fully relaxed* pass (soft branch without the straight-through estimator, temperature-T reads) on the full simulator running Space Invaders, sampled around the two critical boundaries. **Measured** N_{div}: the first instruction (of N=3000 from reset) at which the relaxed trajectory deviates from the executed (straight-through) one; "exact" means no deviation over the whole run. **Predicted** (cast-margin model): per-branch exactness $p_{\text{branch}}(\alpha)$, per-read exactness $p_{\text{read}}(T)$, and the per-step likelihood $P_{\text{step}} = p_{\text{read}}^{\rho}\, p_{\text{branch}}^{f_b}$ ($\rho = 2.16$, $f_b = 0.374$); the forward is bit-exact iff $P_{\text{step}} = 1$. JAXTARI uses the identical cast-to-Int logic and is omitted.

		Meas.	Predicted		
α	T	N_{div}	p_{branch}	p_{read}	P_{step}
Branch boundary (T=0.14, read-exact)					
2	0.14	7	0.039	1.000	0.298
3	0.14	879	0.953	1.000	0.982
4	0.14	1121	0.988	1.000	0.996
5	0.14	1216	0.996	1.000	0.999
6	0.14	*exact*	1.000	1.000	1.000
Temperature boundary (α=6, branch-exact)					
6	0.08	*exact*	1.000	1.000	1.000
6	0.10	*exact*	1.000	1.000	1.000
6	0.12	*exact*	1.000	1.000	1.000
6	0.14	*exact*	1.000	1.000	1.000
6	0.15	803	1.000	0.981	0.959
6	0.18	2	1.000	0.223	0.039
6	0.20	2	1.000	0.144	0.015

Figure 2 maps this likelihood across the whole (α, T) plane: a bit-exact corner at large α and small T, a gradual branch boundary, and a sharp temperature boundary. Table 2 zooms into the two boundaries. As the faithful measurement we report the first instruction at which the relaxed trajectory deviates from the executed one over $N = 3000$ steps ("exact" means no deviation): the game re-initialises much of its state every frame, so a diverged run can partially resynchronise and an end-of-run state match would overstate exactness. Measurement and model agree—the forward is bit-exact precisely where $P_{\text{step}} = 1$, namely $\alpha \geq 6$ (so τ_b exceeds the largest offset, 122) and $T \leq 0.14$, and the first-divergence step grows as $P_{\text{step}} \to 1$ (α=3, 4, 5 at T=0.14 first diverge at steps 879, 1121, 1216). The branch boundary is gradual in α while the temperature boundary is sharp: a single max-amplitude neighbour already pulls a read by $\approx 1.7 > \frac{1}{2}$, so p_{read} drops from 1 at $T \leq 0.14$ to 0.14 at $T = 0.20$. This is the empirical face of Theorem 2: exactness is recovered as $\alpha \to \infty$ and $T \to 0$. The JAXTARI twin uses the identical cast-to-Int logic, so the same analysis applies.

Choosing α and T. The two bounds are *program-independent*, which is what makes the setting reliable across

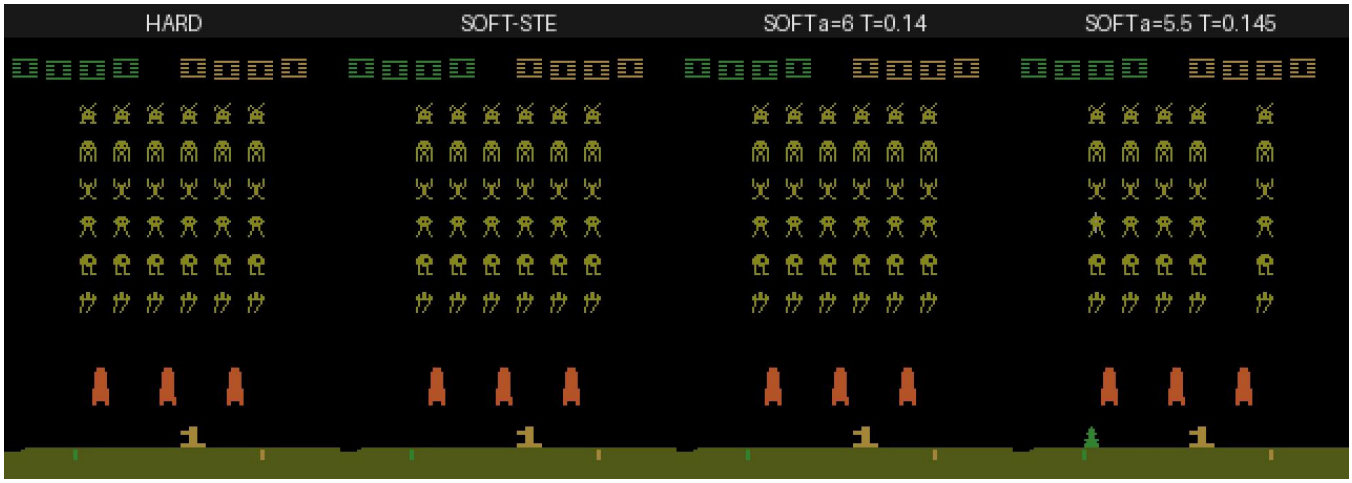

Figure 3: One frame of the 60-second full-simulator comparison (Space Invaders, beam-timed). **HARD** and **SOFT-STE** are pixel-identical (Theorem 1). **SOFT** $\alpha{=}6, T{=}0.14$ is inside the bit-exact corner and stays identical to HARD for the whole rollout. **SOFT** $\alpha{=}5.5, T{=}0.145$ is just past the read boundary and diverges *mildly*: the game stays recognisable but a column of invaders is missing—the localised effect of one corrupted sprite-position read, persistent over the rollout.

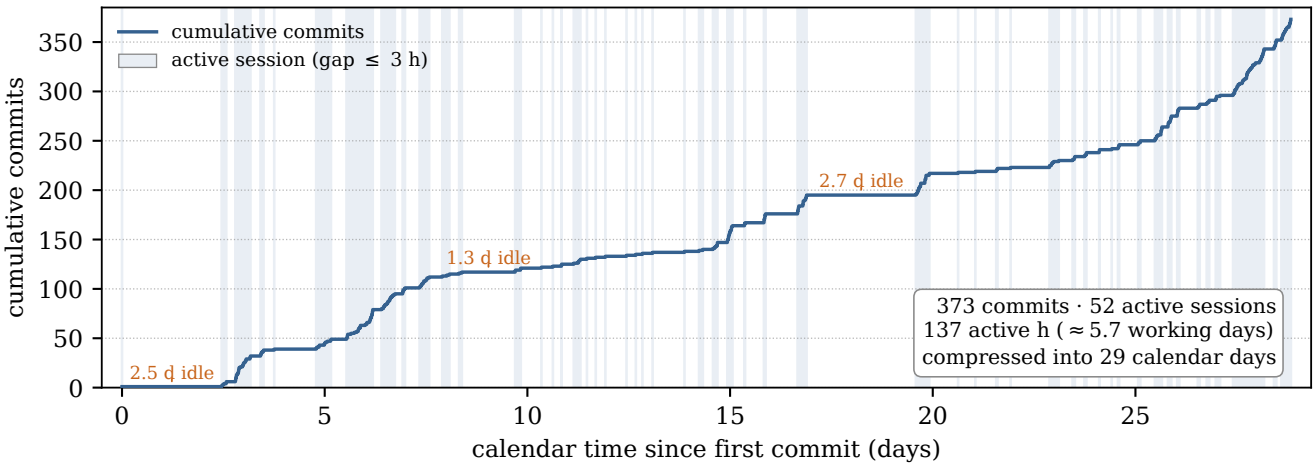

Figure 4: Implementation effort reconstructed from the version-control log. The step curve is the cumulative number of commits against calendar time, and each shaded band is an active work session (a maximal run of consecutive commits no more than three hours apart), and the long flat stretches between bands are idle gaps, the largest of which are annotated. The 373 commits form 52 active sessions totalling $\sim$137 hours of work—about 5.7 round-the-clock days—spread across a 29-day calendar window.

Table 3: Per-game conformance over the 64 ALE games supported by our XITARI configuration. **Map**: mapper auto-detected by the ports (the set exercises seven of the ten implemented formats: 2K, 4K, F8, F6, E0, FE, F8SC; F4, F6SC and F4SC are implemented but unused here). **SHA-256**: first 12 hex digits of the ROM image (full hashes ship in the repository manifest; ROMs are not redistributed). **R/P**: RAM- and pixel-exact against XITARI on every evaluated frame (30 NOOP frames for RAM, 60 frames of the fixed action stream for pixels, after the standard 60-NOOP+4-reset boot). All 64 games are exact in both (✓); JAXTARI matches JUTARI game-for-game.

Game	Map	SHA-256	R	P
air_raid	4K	6b5a22ec074a	✓	✓
alien	4K	9cd556c7d7f5	✓	✓
amidar	4K	da5b760d3ada	✓	✓
assault	4K	d32ca4dd9e84	✓	✓
asterix	F8	b6161db0530d	✓	✓
asteroids	F8	d1a6e808412b	✓	✓
atlantis	4K	cef4b7b91ea2	✓	✓
bank_heist	4K	c32daffdb926	✓	✓
battle_zone	F8	efd6f639e724	✓	✓
beam_rider	F8	83b30cb52649	✓	✓
berzerk	4K	cdff0422329d	✓	✓
bowling	2K	dff44a85289f	✓	✓
boxing	2K	54760d7e0710	✓	✓
breakout	2K	376323f051c3	✓	✓
carnival	4K	80eab759b6bd	✓	✓
centipede	F8	6def669f54bd	✓	✓
chopper_command	4K	055637282252	✓	✓
crazy_climber	F8	486ad4cc8956	✓	✓
defender	4K	e3c7ed7a073f	✓	✓
demon_attack	4K	8d72feeb267d	✓	✓
double_dunk	F6	c0ea53a91a39	✓	✓
elevator_action	F8SC	f3b3ad03fd8c	✓	✓
enduro	4K	ca9bb89755a6	✓	✓
fishing_derby	2K	84753bc6ecb4	✓	✓
freeway	2K	3ef620234b98	✓	✓
frostbite	4K	cbfdad89480d	✓	✓
gopher	4K	b4aff03aeb0f	✓	✓
gravitar	F8	17a19cd7b5ca	✓	✓
hero	F8	75bd02ec7545	✓	✓
ice_hockey	4K	06df1fc568d9	✓	✓
jamesbond	E0	995253f0d3a2	✓	✓
journey_escape	4K	963e2d3da52b	✓	✓
kangaroo	F8	a2826eaea3a8	✓	✓
krull	F8	b40f10d14217	✓	✓
kung_fu_master	F8	3f6501a649ad	✓	✓
montezuma_revenge	E0	69d363a74754	✓	✓
ms_pacman	F8	dde0b43c5dee	✓	✓
name_this_game	4K	cc219a7246e8	✓	✓
pacman	4K	58e781b472e0	✓	✓
phoenix	F8	b01085bc5227	✓	✓
pitfall	4K	c719b47714d8	✓	✓
pong	2K	41623e3c2614	✓	✓
pooyan	4K	ac9abbfc272d	✓	✓
private_eye	F8	c8f9ce1b2e80	✓	✓
qbert	4K	3257221832a7	✓	✓
riverraid	4K	4c6842b8af64	✓	✓
road_runner	F6	4b6c2e68d693	✓	✓
robotank	FE	c186409481e6	✓	✓
seaquest	4K	fbc29f4678f6	✓	✓
skiing	2K	804eff5f0196	✓	✓
solaris	F6	0afa36e5f4d8	✓	✓
space_invaders	4K	7224b17462b9	✓	✓
star_gunner	4K	3ad501a39280	✓	✓
surround	2K	c6864f5c9f43	✓	✓
tennis	2K	5a2052f020bf	✓	✓
time_pilot	F8	8ea97e335a2f	✓	✓
tutankham	E0	403397fe8583	✓	✓
up_n_down	F8	74890a43e591	✓	✓
venture	4K	1870b0dd6fb1	✓	✓
video_pinball	4K	cef82c39bbb0	✓	✓
videochess	4K	d44f2a115393	✓	✓
wizard_of_wor	4K	d69ee89e65c3	✓	✓
yars_revenge	4K	ff777c8d4ea0	✓	✓
zaxxon	F8	e1c323ce00cc	✓	✓

games. A 6502 branch displacement is at most $|\delta| \le 127$, so $\frac{1}{2}(1+e^{\alpha}) > 127$—that is $\alpha \ge \ln(2\cdot 127-1) \approx 5.5$—makes every branch round correctly, and $\alpha \ge 6$ suffices. A read is pulled away from its addressed byte by at most $255\,(1-w_0) \approx 510\,e^{-1/T}$, which stays below $\frac{1}{2}$ once $T \le 1/\ln(4\cdot 255) \approx 0.144$; the corner value $T{=}0.14$ sits just inside this (worst-case pull $\approx 0.40 < \frac{1}{2}$). Both bounds depend only on the instruction set and the 8-bit byte range, not on the program, so $\alpha{=}6,\ T{=}0.14$ is bit-exact across games—verified over 6000 instructions on Pong, Breakout and Space Invaders—whereas *just outside* them the forward fails on a game-dependent subset ($\alpha{=}5$ diverges on Space Invaders but not Pong or Breakout; $T{=}0.15$ diverges on Pong and Space Invaders but not Breakout), which is exactly why the conservative bounds are needed. Within the exact region the relaxed gradient is strongest at the boundary (it vanishes as $\alpha \to \infty, T \to 0$; Theorem 2), so the recommended operating point is the corner of the bit-exact region, $\alpha{=}6,\ T{=}0.14$ (Figure 2): the smallest α and largest T that keep the forward bit-exact, giving the most informative gradient compatible with exactness. We present this as a conservative, program-independent operating point—its per-cast bounds are worst-case over the instruction set and the 8-bit byte range, and the cross-game exactness is confirmed empirically (Pong, Breakout, Space Invaders)—not as a theorem that every program is exact at this setting.

Full-simulator demonstration. The per-cast model is borne out on a real 60-second rollout. We render Space Invaders beam-timed in four modes side by side (Figure 3): the exact hard run, the executed soft (straight-through) run, and the fully relaxed run at two settings. The relaxed frames come from the same cycle-accurate hard renderer driven through a default-off relaxation hook on the CPU's branch and reads, so HARD, SOFT-STE and any in-corner setting render pixel-identically. At $\alpha{=}6,\ T{=}0.14$—inside the bit-exact corner—the relaxed run stays identical to HARD for the entire rollout. At $\alpha{=}5.5,\ T{=}0.145$—just past the read boundary—it diverges, but only mildly: the game remains recognisable with a persistent, localised error (a depleted column of invaders), because the relaxation corrupts a sprite-position read rather than the control flow.

Delayed divergence. A setting can also boot cleanly and diverge only after many frames—the Bernoulli first failure deep into the rollout. The mechanism is that the read margins are *time-varying* for RAM: a data read is exact while its neighbouring bytes are similar and crosses $\frac{1}{2}$ only once the game state evolves into a high-contrast layout, so the most-borderline cast may be one the program reaches only late. Sweeping T just below the read boundary on a long clean-boot rollout (boot exact, relaxation enabled only for the rollout) makes the first-divergence frame recede: at $\alpha{=}20$ the rollout first deviates on frame 1 for $T{=}0.1448$, on frame 194 for $T{=}0.1446$, and only on frame 2365 ($\approx$40 s of play) for $T \le 0.1444$—reproducing the game bit-for-bit for tens of seconds before a single late RAM read first rounds wrong. (Below $T{=}0.1465$, the lowest fixed ROM-read threshold, no opcode/operand fetch ever fails, so this late divergence is driven purely by an evolving RAM read.) The exact–diverge boundary is thus a per-cast threshold *spectrum* over both fixed (ROM) and state-dependent (RAM) reads, not a single cliff.

5 Per-Game Conformance

Table 3 lists every game in the conformance set with its auto-detected mapper, a short ROM hash, and its per-frame RAM and pixel exactness, so the headline 64/64 claim can be audited game by game. Both ports pass every game; JAXTARI and JUTARI agree game-for-game, and the cross-check between them is part of the test suite.

6 Throughput

We measure soft-mode throughput as steps per second (one step = one CPU instruction) on a fixed ROM after just-in-time warm-up, on a single Apple M1 Max core (JAX CPU backend, Julia 1.12), reporting the median of repeated runs. Table 4 summarises the three operating points the rest of this section develops; the per-batch scaling behind the batched rows is in Tables 5 (CPU) and 6 (GPU).

Table 4: Throughput across engine × backend (real Pong ROM, soft mode, 3,000-step rollout, median of repeated runs after JIT warm-up). JUTARI (Julia) executes one environment with inlined opcode handlers; JAXTARI (JAX) is slow per single environment but recovers—and on the GPU far exceeds—that by `vmap`-batching the compiled rollout, its intended regime. Single-env rows are steps/s (= env-steps/s at batch 1); batched rows are aggregate env-steps/s at their best batch ($N{\approx}4096$ on the GPU).

engine & backend	mode	env-steps/s
JUTARI, CPU	single env	370,100
JAXTARI, CPU	single env	1,178
JAXTARI, CPU	batched (`vmap`)	∼60,000
JAXTARI, GPU	batched (`vmap`)	**3,122,386**

At single-instruction granularity JUTARI sustains ∼370,100 steps/s and JAXTARI ∼1,178, a ∼314× gap. The difference is dominated by JAX's per-operation kernel-launch overhead: each soft step dispatches one opcode through a 256-way switch, so at single-instruction granularity the launch overhead dwarfs the instruction, whereas Julia inlines the small handlers—a property of the execution granularity, not the AD systems. For a gradient-based XAI workflow (one forward-plus-backward pass per attribution) JAXTARI's ∼42 s per 50,000-step trace is comfortably interactive.

Two modes recover throughput beyond this single-step figure: compiling a whole rollout into one `lax.scan` (∼30,000 steps/s for a single environment, a ∼25× gain that amortises the launches), and `vmap`-ing that scan over a batch of independent environments. Table 5 reports the batched scaling on the CPU backend (Apple M1 Max, real Pong ROM, median of repeated 3,000-step runs after JIT warm-up; the batched run is verified bit-identical to the unbatched one). Aggregate throughput rises *sub-linearly* with the batch because the data-dependent 256-way opcode

dispatch (`lax.switch`) evaluates all candidate handler branches under `vmap`; on a CPU these branches serialise, so the aggregate asymptotes near ∼60,000 env-steps/s (Table 5).

GPU. On a GPU the same all-branch evaluation maps onto the device's parallel lanes instead of serialising, so batching the differentiable rollout is the intended operating regime. Table 6 and Figure 5 measure this on a single *commodity* GTX 1080 Ti (an 11 GB Pascal card; real Pong ROM, a 3,000-step `lax.scan` `vmap`-ped over N environments, median of 3 runs after JIT warm-up, verified bit-identical to the CPU rollout). Aggregate forward throughput climbs from ∼2,500 env-steps/s at $N{=}1$ to a peak of ∼2.95M at $N{\approx}4096$—roughly $50\times$ the CPU asymptote and $\sim 1200\times$ the single-environment rate—then rolls off gently as the device saturates. The workload is compute/occupancy-bound, not memory-bound: the peak uses only ∼0.1 GB, so the ceiling is lane parallelism rather than capacity. Crucially the reverse-mode gradient is nearly free at scale—*forward+gradient* tracks forward to within ∼5% at the peak (∼2.8M env-steps/s)—so a commodity GPU turns batched *differentiable* rollouts (gradient-based RL or attribution over thousands of environments at once) into the natural way to use JAXTARI. A second, newer architecture (Quadro RTX 5000, 16 GB Turing) traces the same curve and peaks slightly higher (∼3.12M env-steps/s at $N{\approx}4096$; Figure 5), so the scaling reflects the batched all-branch execution rather than one specific GPU. We report widely-available cards deliberately: the curve's shape, not a datacenter accelerator, is the point.

Table 5: Batched soft-mode throughput of JAXTARI on the CPU backend (M1 Max, Pong, a compiled `lax.scan` `vmap`-ped over N environments). *Aggregate* is total environment-steps per second; *per-env* is aggregate/N. Batching trades single-environment latency for aggregate throughput.

batch N	aggregate (env-steps/s)	per-env (steps/s)	vs. $N{=}1$
1	30,200	30,200	1.0×
16	35,200	2,200	1.2×
64	48,300	750	1.6×
128	55,900	440	1.9×

7 Implementation Effort

Figure 4 is the evidence behind the effort estimate in the main paper. All effort numbers are derived from the project's `git` history, which—like the rest of the artifact—will be made public on acceptance of the paper, so every figure here can be independently recomputed from the commit log. Because every step of the project was committed, the commit timestamps record when work actually happened. To turn them into an active-time figure we group the commits into sessions, treating a gap of more than three hours between consecutive commits as a session boundary. Three hours is well above the in-session rhythm—the median gap between consecutive commits is about 13 minutes—and well below the multi-hour and overnight gaps that separate genuine work periods, so the partition is insensitive to the exact threshold: the active total is 106 hours at a two-hour cutoff and 162 hours at a four-hour cutoff, bracketing the 137 hours obtained at three hours.

With the three-hour boundary the 373 commits fall into 52 sessions whose durations sum to 136.8 hours, i.e. about 5.7 round-the-clock days (the coding agents run continuously, not in eight-hour shifts), even though they are spread over a 29-day calendar window. The remaining ∼80% of the calendar span is idle and excluded. It consists mostly of stretches during which the lead programmer was travelling and without reliable internet access, which appear in the plot as the long flat segments. This active total—not the calendar span—is what we compare against the scale of the reference codebase in the main paper.

Table 6: Batched soft-mode JAXTARI throughput on a single *commodity* GTX 1080 Ti (11 GB Pascal), Pong, a 3,000-step `lax.scan` `vmap`-ped over N environments (median of 3 runs, JIT warm-up excluded). Aggregate environment-steps per second, forward and forward+gradient. Both peak near $N{=}4096$ at ∼3M env-steps/s—$\sim 50\times$ the CPU `vmap` asymptote (Table 5)—and the gradient costs only ∼5% more.

batch N	forward (env-steps/s)	forward+grad (env-steps/s)
1	2,459	1,275
64	259,705	225,371
256	677,888	596,736
1024	2,024,866	1,830,998
4096	**2,953,695**	**2,805,168**
16384	2,850,165	2,577,024
65536	2,599,999	2,533,964

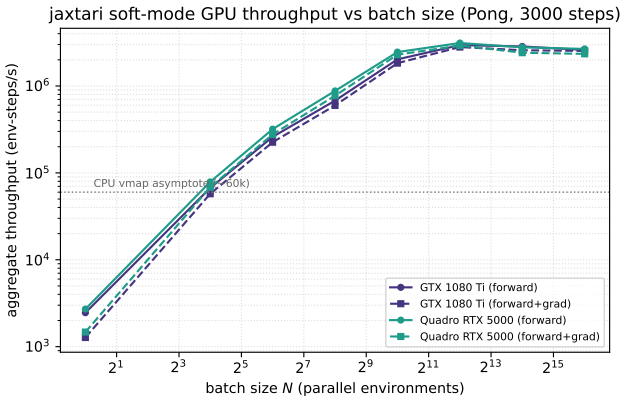

Figure 5: JAXTARI soft-mode GPU throughput vs. batch size N (Pong, 3,000 steps; two commodity GPUs, GTX 1080 Ti and Quadro RTX 5000). Forward and forward+gradient both rise $\sim$1200$\times$ from N=1 to a peak near N=4096 ($\sim$3M env-steps/s, $\sim$50$\times$ the CPU `vmap` asymptote, dotted), then roll off as the device saturates—the 256-way `lax.switch` all-branch dispatch parallelises across lanes on the GPU where it serialises on the CPU.

References

Goldberg, D. 1991. What Every Computer Scientist Should Know About Floating-Point Arithmetic. *ACM Computing Surveys*, 23(1): 5–48.

IEEE. 2019. IEEE Standard for Floating-Point Arithmetic (IEEE Std 754-2019). IEEE Std 754-2019.